CFR-Poker GUI

Visualizer + Multiplayerdemo für CFR Poker

Modul: Advanced Python Wise25/26
Dozent: Markus Schubert

Bachelorarbeit

- CFR lernt Strategien für Poker Environments
- Trainierte Strategie = Dictionary:
    - Key: Information Set / Spielzustand
    - Value: Wahrscheinlichkeiten für legale Aktionen
- StrategyAgent sampelt daraus die nächste Aktion
- Game Enviroment verwaltet Legale Aktionen in einem bestimmten Infoset, Pot, Runden, Deck, Hand Evaluation, ...

Ziel dieses Projekts

- Ein GUI für mehrere Varianten (Kuhn → Leduc → Twelve Card Poker → Rhode → Royal → Limit)
- Normalisierung: GUI vereinheitlicht unterschiedliche Env States (z.B. `public_card` vs `public_cards`, `player_bets` vs `total_bets`)
- Drei Nutzungsfälle:
    - Agent vs Human (Gegen eine trainierte Strategie spielen)
    - Human vs Human (online spielen)
    - Agent vs Agent (Visualizer / Replay)

Aufbau

- `layouts/*`: nur Anordnung (Table, Player, History, Control Area)
- `agent_vs_human.py` / `human_vs_human.py`: Logik + State Updates
- `components/*`: PokerTable, PlayerWidgets, VisualCard, PotDisplay+Chips, HistoryView, ActionButtons, PlaybackControls
- `audio/*`: Action-Sounds
- `runner/*`: CLI-Argumente parsen

Layout (Agent/Human vs Human)

Hierarchische Struktur:

- Game Area: `QHBoxLayout` (horizontal, teilt Links/Rechts)
  - Links: `QVBoxLayout` (vertikal)
    - Player Top → Table → Player Bottom → Controls
  - Rechts: History View (ausklappbar)
- Table: Community Cards (`QHBoxLayout`), Pot Display, Chips
  - Pot: Label zeigt Wert + Chips zufällig angeordnet
- Player Widgets: Karten horizontal (`QHBoxLayout`), Name, Hand Strength

Grafische Komponenten

- **Karten**: `QLabel` rendert HTML-Code → Karte wird mit HTML/CSS "gemalt" (Rank/Suit Layout, Unicode-Symbole ♠ ♥ ♦ ♣, Farben rot/schwarz)
- **Verdeckte Karten**: `QLabel` mit CSS-Styling (blauer Hintergrund #1a5490)
- **Tisch**: `QPainter` zeichnet grünen Poker-Tisch (`drawRoundedRect`)
- **Chips**: `QPainter` zeichnet Ellipsen (Ring + innerer Kreis)

Modus 1: Agent vs Human

- GUI hält ein lokales `game` Objekt
- Human klickt → GUI prüft `legal_actions` → `game.step(action)` → UI Update
- Agent Zug → `StrategyAgent` sampelt Aktion aus Strategie (Dictionary: InfoSet → Wahrscheinlichkeiten) → `game.step(action)` → UI Update
- Am Ende werden Karten aufgedeckt, Gewinner/Payoff steht im History View

Modus 2: Human vs Human

- Server = Game Authority, Clients = View + Input
- **REST API**: Architekturstil für Client-Server-Kommunikation über HTTP; Server stellt Endpoints bereit, Client sendet Anfragen (GET = Daten abrufen, POST = Aktion ausführen)
- **Flask**: Python-Framework zum einfachen Erstellen von REST APIs; Client sendet HTTP-Anfragen (GET/POST); **QTimer** für Polling alle 100ms
- Endpoints: `GET /player_id` (Slot 0/1), `GET /state` (State + private_cards), `POST /action`, `POST /reset`, `POST /disconnect`
- Thread-Safety: Flask threaded Server (mehrere Clients gleichzeitig) + Lock um zentrales `game` Objekt (verhindert Race Conditions)

Ausblick

- Agent vs Agent Modus
- Optional: Strategietipps im UI
- Multiplayer ausserhalb des selben Netzwerks, Scoreboard, begrenzter Chip Stack, No Limit Holdem, Über UI zu Server verbinden anstatt Terminal